

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I D.HUSSAINIAIAH(192425300) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.HUSSAINIAIAH

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints: A. Limited processing power (mobile CPU)

B. Real-time performance

C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

Answer:

1. Input Acquisition

Capture frames from the front camera using `cv2.VideoCapture` at a reduced resolution (e.g., 160x120 or 240x180) to minimize the data volume the CPU must process per frame.

2. Preprocessing

Convert each frame to grayscale (`cv2.cvtColor`) to cut computation to one-third, then apply histogram equalization (`cv2.equalizeHist`) or CLAHE to normalize brightness so that changing lighting conditions do not affect downstream detection.

3. Face Detection

Use a Haar Cascade or LBP cascade classifier (`cv2.CascadeClassifier`) rather than a deep-learning detector. Cascade detectors are computationally cheap and run comfortably on a mobile CPU while still being accurate enough for a single frontal face.

4. Lightweight Tracking Between Detections

Instead of running full face detection on every frame, run detection every N frames and track the face region in between using a lightweight tracker (e.g., KLT optical flow or a MOSSE correlation tracker). This keeps the pipeline robust to slight head movement without repeating an expensive detection step.

5. Feature Extraction

Extract handcrafted features such as Local Binary Patterns (LBP) or HOG from the facial ROI. These features are inexpensive to compute and are known to be robust to minor illumination and pose variation, which suits the given constraints better than deep CNN features.

6. Expression Classification

Feed the extracted features into a small, pre-trained classifier (e.g., an SVM or a shallow/quantized CNN) trained on the three target classes (happy, sad, neutral). A lightweight classifier keeps inference time low.

7. Output/Display

Draw the bounding box (`cv2.rectangle`) and overlay the predicted expression label (`cv2.putText`) on the live camera preview for the user.

Justification – Balancing Accuracy and Efficiency

The pipeline deliberately avoids heavy deep-learning detectors and instead combines cascade-based detection with tracking, and handcrafted features with a lightweight classifier. Detecting only every few frames and tracking in between drastically reduces CPU load while equalization and LBP/HOG features preserve robustness to lighting changes and small head movements, together satisfying real-time performance on limited mobile hardware without a significant loss in accuracy.

2. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Answer:

1. Video Stream Handling

Use `cv2.VideoCapture` to read the live feed continuously inside a loop, with frame resizing and optional frame-skipping to bound the amount of processing per second and prevent buffer lag on long-running streams.

2. Preprocessing

Convert frames to grayscale and apply Gaussian blur (`cv2.GaussianBlur`) to suppress camera noise before background modelling.

3. Vehicle Detection

Apply a background subtractor (`cv2.createBackgroundSubtractorMOG2`) to isolate moving foreground objects, then use `cv2.findContours` with a minimum-area filter to identify candidate vehicle blobs. For higher accuracy a pre-trained lightweight vehicle/Haar cascade or MobileNet-SSD (via `cv2.dnn`) can be layered on top of the candidate regions.

4. Tracking

Assign a unique ID to each detected vehicle using a centroid-tracking algorithm (or a Kalman-filter-based tracker such as SORT) so that the same vehicle is followed across consecutive frames rather than being re-counted.

5. Counting

Define a virtual counting line/ROI across the road. When a tracked vehicle's centroid crosses this line for the first time, increment the count and mark that vehicle as already counted to avoid duplicate counting.

6. Visualization

Draw bounding boxes and IDs on each vehicle (`cv2.rectangle`, `cv2.putText`), draw the counting line, and overlay running statistics (count, average speed if required) on the frame using `cv2.imshow` or a stream server.

7. Continuous Operation

Wrap the capture loop in error handling that automatically reconnects on stream drop, and keep per-frame operations lightweight (background subtraction + contour filtering) so the system keeps pace with the incoming stream.

Justification – Accuracy and Real-Time Performance

Background subtraction and contour analysis are computationally cheap and well suited to continuous streams, while the Kalman/centroid tracker maintains vehicle identity across frames, which is what makes the counting logic accurate rather than merely detecting objects. The counting line design specifically prevents double counting, and frame-skipping/resizing keeps end-to-end delay minimal, satisfying the continuous, low-latency requirement.

3. Real-Time Object Detection on Mobile Device

A mobile application must:

- A. Detect objects using the camera
- B. Run in real time
- C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

Answer:

1. Image Acquisition

Capture frames via `cv2.VideoCapture` at a reduced resolution suited to the target model's input size (e.g., 300x300), minimizing memory footprint from the start.

2. Preprocessing

Resize and normalize the frame, then convert it into a network-ready blob using `cv2.dnn.blobFromImage`.

3. Detection

Load a lightweight, quantized model such as MobileNet-SSD or a Tiny-YOLO variant through OpenCV's DNN module (`cv2.dnn.readNet`), which is optimized for CPU inference without needing a separate deep-learning framework. Run inference on every frame, or every 2nd–3rd frame if the CPU cannot sustain full-frame-rate inference.

4. Post-processing

Filter low-confidence detections and remove overlapping boxes using Non-Maximum Suppression (`cv2.dnn.NMSBoxes`).

5. Threading

Run frame capture on one thread and model inference/display on another so that a slow inference step does not block frame acquisition, keeping the interface responsive.

6. Display

Draw the detected bounding boxes and class labels (`cv2.rectangle`, `cv2.putText`) and overlay an FPS counter to monitor real-time performance.

Justification – Efficiency Under Constraints

Using a mobile-optimized quantized model through cv2.dnn avoids the memory overhead of full-scale deep networks, while reduced input resolution, frame-skipping, and multithreaded capture/inference keep both CPU utilization and RAM usage low. NMS keeps the output clean without added computational cost, together meeting the real-time requirement on limited mobile hardware.

4. Smart Attendance System using Face Recognition

A system must:

- A. Capture faces from a webcam
- B. Recognize registered users
- C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

Answer:

1. Face Capture

Continuously read frames from the webcam using `cv2.VideoCapture` and detect faces with a Haar Cascade or DNN-based face detector.

2. Preprocessing

Align, crop, resize, and normalize each detected face region so that every input to the recognizer is in a consistent format regardless of position in frame.

3. Feature Extraction and Recognition

Generate a facial descriptor for the detected face — either using `cv2.face.LBPHFaceRecognizer` or embeddings from a pre-trained embedding network — and compare it against the registered-user dataset using a distance metric. Only accept a match if the similarity score crosses a strict, pre-tuned threshold.

4. False-Recognition Safeguards

Require the same identity to be matched consistently across several consecutive frames (temporal voting) before accepting it, and add a basic liveness check (e.g., eye-blink detection) to prevent spoofing with a static photo. Any match below the confidence threshold is labelled 'Unknown' rather than force-matched to the closest registered user.

5. Marking Attendance

Once a user is confidently recognized, log their ID with a timestamp into a CSV file or database, guarding against duplicate entries by checking whether that user has already been marked for the current session/day.

6. Feedback

Display the recognized name and a confirmation message on-screen so the user knows attendance was registered.

Justification – Avoiding False Recognition

The combination of a strict distance/confidence threshold, multi-frame temporal confirmation, and a liveness check specifically targets the two main sources of false recognition — look-alike false positives and photo/video spoofing — while the duplicate-entry check ensures attendance integrity, together giving a reliable end-to-end recognition pipeline.

5. Motion Detection for Home Security

A home security system must:

- A. Detect motion from a camera feed
- B. Trigger alerts only for significant movement
- C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

Answer:

1. Video Capture

Continuously read frames from the security camera using `cv2.VideoCapture`.

2. Preprocessing

Convert each frame to grayscale and apply Gaussian blur (`cv2.GaussianBlur`) to suppress sensor noise before comparing frames.

3. Motion Detection

Use an adaptive background subtractor (`cv2.createBackgroundSubtractorMOG2`, which models gradual background changes such as lighting drift or moving foliage) rather than simple frame differencing, so that slow scene changes are absorbed into the background model instead of triggering false motion.

4. Thresholding and Morphology

Threshold the foreground mask (`cv2.threshold`) and clean it with morphological dilation/erosion (`cv2.dilate`, `cv2.erode`) to remove speckle noise and merge fragmented regions belonging to the same moving object.

5. Contour Filtering

Extract contours (`cv2.findContours`) and discard any whose area is below a minimum-area threshold, filtering out small movements caused by leaves, shadows, or camera noise.

6. Temporal Confirmation

Only raise an alert if a qualifying motion region persists across several consecutive frames rather than appearing once, further reducing spurious single-frame false alarms.

7. Visualization and Alerting

Draw a bounding rectangle around the confirmed motion region (`cv2.rectangle`), timestamp and log the event, and trigger the alert/notification.

Justification – Handling Noise and Background Variation

An adaptive background model tolerates gradual background variation (e.g., changing daylight) instead of flagging it as motion, while the area filter and multi-frame temporal check together eliminate small, one-off disturbances such as sensor noise, insects, or shadows — meaning the system only escalates alerts for genuinely significant, sustained movement.